

# Week 4 — Adders, Selection, and Memory

---

**Target time:** approximately 3 hours

- Whiteboard derivation: about 75 minutes
- Digital Circuits Lab: about 75 minutes
- Draw/build evidence and explanation from memory: about 30 minutes

Gates can transform bits. But a useful machine must do three more specific things: calculate a result, choose which result matters, and keep that result after the inputs move on.

That is this week's path:

gates → adder → selector → stored state → register-transfer datapath

The previous lesson ended with  $\text{Sum} = A \text{ XOR } B$  and  $\text{Carry} = A \text{ AND } B$ . We will rebuild that result rather than merely inherit it, repair its missing carry-in, connect stages into a four-bit adder, then ask two questions that arithmetic alone cannot answer:

1. If several circuits produce values, which one reaches the destination?
2. If the inputs disappear, what keeps the chosen value available?

Those questions lead naturally to multiplexers and storage. They are also the first recognizable pieces of a CPU datapath.

---

## 1. Why engineers needed these circuits

Mechanical calculators used gears and wheels to embody place-value arithmetic. One wheel passing its final position pushed a carry into the next wheel. The rule was visible, but moving parts imposed friction, wear, size, and speed limits.

Relay calculators replaced much of that motion with electrically controlled contacts. Relays could implement Boolean relationships, but their armatures still moved. Vacuum tubes and then transistors removed the mechanical contact from the switching path. Integrated circuits made it practical to repeat a small logic pattern thousands, then millions, then billions of times.

The engineering need did not change:

represent a digit  
→ combine digits according to arithmetic  
→ carry information between positions  
→ choose an operation or source  
→ hold intermediate results  
→ coordinate a sequence of steps

A fast combinational adder by itself is not a programmable computer. It continuously reacts to whatever is on its inputs. It has no opinion about whether its output should be used, and no memory of the previous answer. Selection and storage turn a collection of functions into a datapath that can perform a sequence.

## 2. Derive the half-adder from ordinary addition

Add every possible pair of one-bit unsigned numbers:

| A | B | binary total | Sum | Carry |
|---|---|--------------|-----|-------|
| 0 | 0 | 00           | 0   | 0     |
| 0 | 1 | 01           | 1   | 0     |
| 1 | 0 | 01           | 1   | 0     |
| 1 | 1 | 10           | 0   | 1     |

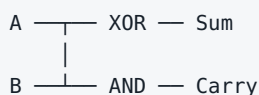
The low result bit is 1 when the inputs differ:

$$\text{Sum} = A \text{ XOR } B$$

The high result bit is 1 only when both inputs are 1:

$$\text{Carry} = A \text{ AND } B$$

Together those gates form a **half-adder**:



Why “half”? It handles the two bits belonging to one column, but not the carry arriving from the column to its right. It works at the least significant bit of a multi-bit addition, where there is normally no incoming carry. Every later column needs a third input.

### Pause and derive

Do not quote the equations. Explain why  $1 + 1$  produces Sum 0 and Carry 1. Then point to the physical output wires on which those two facts would be represented.

## 3. The missing input creates the full-adder

A general bit position receives:

- A, one bit of the first operand;
- B, one bit of the second operand;
- Cin, carry from the less-significant stage.

It produces:

- Sum, the result bit for this position;

- Cout, carry to the more-significant stage.

Write all eight cases:

| A | B | Cin | total | Sum | Cout |
|---|---|-----|-------|-----|------|
| 0 | 0 | 0   | 0     | 0   | 0    |
| 0 | 0 | 1   | 1     | 1   | 0    |
| 0 | 1 | 0   | 1     | 1   | 0    |
| 0 | 1 | 1   | 2     | 0   | 1    |
| 1 | 0 | 0   | 1     | 1   | 0    |
| 1 | 0 | 1   | 2     | 0   | 1    |
| 1 | 1 | 0   | 2     | 0   | 1    |
| 1 | 1 | 1   | 3     | 1   | 1    |

The Sum is 1 when an odd number of inputs are 1:

$$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

Carry-out is 1 when at least two inputs are 1—the majority function:

$$\text{Cout} = (A \text{ AND } B) \text{ OR } (\text{Cin} \text{ AND } (A \text{ XOR } B))$$

Another equivalent form is:

$$\text{Cout} = (A \text{ AND } B) \text{ OR } (A \text{ AND } \text{Cin}) \text{ OR } (B \text{ AND } \text{Cin})$$

## Build it from two half-adders

$$P = A \text{ XOR } B$$

$$C1 = A \text{ AND } B$$

$$\text{Sum} = P \text{ XOR } \text{Cin}$$

$$C2 = P \text{ AND } \text{Cin}$$

$$\text{Cout} = C1 \text{ OR } C2$$

A,B — half-adder — P,C1

P,Cin — half-adder — Sum,C2

C1,C2 — OR — Cout

This is a **full-adder**. Its name does not mean it adds a whole machine word. It means one bit position now has the full interface required to participate in a chain.

## Carry generate and propagate

Two intermediate ideas make wider adders easier to reason about:

```

G = A AND B      this position generates a carry
P = A XOR B      this position propagates an incoming carry
Cout = G OR (P AND Cin)

```

If  $G = 1$ , this position creates a carry regardless of  $Cin$ . If  $P = 1$  and  $G = 0$ , the outgoing carry follows the incoming carry. If both operand bits are 0, the chain stops.

## 4. Ripple-carry: repeat a small correct stage

Connect each  $Cout$  to the next stage's  $Cin$ :

```

              c1          c2          c3
Cin=0 → [FA bit 0] → [FA bit 1] → [FA bit 2] → [FA bit 3] → Cout
      A0 B0      A1 B1      A2 B2      A3 B3
      |         |         |         |
      S0      S1      S2      S3

```

This is a four-bit **ripple-carry adder**. It is regular, easy to build, and correct after its signals settle.

The qualification matters. Gates have propagation delay. Stage 1 cannot know its final  $Cin$  until stage 0 produces  $c1$ ; stage 2 may wait for  $c2$ , and so on. A carry may ripple from the least significant position through every stage. Intermediate sum bits can briefly show non-final values.

For example:

```

  0111
+ 0001
-----
 1000

```

The low position generates a carry. At each following position,  $A \oplus B = 1$ , so the carry propagates through. In a simple timing model, the most significant answer cannot be trusted until that path settles.

This is why wider arithmetic creates a timing problem. A 64-bit ripple-carry adder is not wrong; its worst-case path is simply long. Faster designs predict or combine carry information using carry-lookahead, carry-select, prefix, or other structures. Those designs spend more circuitry and wiring to reduce delay. The deep trade-off is:

```

simple repeated structure ↔ shorter critical path

```

## 5. Trace 3 + 5 all the way to 8

The tracker checkpoint asks more than binary arithmetic. We must cross every layer.

### Meaning and encoding

We choose unsigned four-bit binary:

```
3 → 0011
5 → 0101
8 → 1000
```

That mapping is an encoding convention. The wires themselves do not carry “three” or “five.”

## Logical inputs

Write bits least significant first for the stage trace:

```
A = 0011 → A3 A2 A1 A0 = 0 0 1 1
B = 0101 → B3 B2 B1 B0 = 0 1 0 1
Cin to bit 0 = 0
```

| bit | A | B | Cin | Sum | Cout | reason                              |
|-----|---|---|-----|-----|------|-------------------------------------|
| 0   | 1 | 1 | 0   | 0   | 1    | two ones generate carry             |
| 1   | 1 | 0 | 1   | 0   | 1    | operand difference propagates carry |
| 2   | 0 | 1 | 1   | 0   | 1    | operand difference propagates carry |
| 3   | 0 | 0 | 1   | 1   | 0    | incoming carry becomes this sum bit |

Read S3 S2 S1 S0 :

```
10002 = 8
```

## Physical implementation

In one possible CMOS implementation:

1. Driver circuits establish voltages on eight operand wires.
2. Receivers classify those voltages as logical 0011 and 0101.
3. Transistor networks arranged as XOR, AND, and OR gates respond locally.
4. Internal wire voltages representing carry change and propagate from bit 0 toward bit 3.
5. After the worst-case combinational delay, output wires settle to voltage ranges classified as 1000.
6. A destination register may sample those outputs at an allowed clock edge.
7. Under the chosen unsigned interpretation, software or hardware treats 1000 as eight.

The arithmetic is our abstract description of that causal physical behavior. No transistor sees the symbols 3, +, 5, or 8.

## What the clock did not do

The clock did **not** cause every transistor in the adder to switch. The combinational gates respond whenever their input voltages change, and many transistors do not change state for a particular operation. In a synchronous datapath, the clock defines when storage elements are permitted to capture a settled result. It provides timing discipline, not a universal command that all transistors toggle.

## 6. Selection: one destination, several possible sources

Suppose an ALU has already computed:

```
A AND B
A OR B
A + B
```

Connecting all three outputs directly to the same destination wire would create electrical contention, not a useful choice. The datapath needs a controlled selector.

### The 2-to-1 multiplexer

A **multiplexer**, or mux, chooses one data input:

```
S = 0 → Y = D0
S = 1 → Y = D1
```

| S | D0 | D1 | Y |
|---|----|----|---|
| 0 | 0  | X  | 0 |
| 0 | 1  | X  | 1 |
| 1 | X  | 0  | 0 |
| 1 | X  | 1  | 1 |

X means “do not care for this row,” not an unknown electrical state.

Derive the expression:

```
Y = (NOT S AND D0) OR (S AND D1)
```

When  $S=0$ , the first path is enabled and the second suppressed. When  $S=1$ , the roles reverse. A word-wide mux repeats that one-bit selection for every bit while sharing the select signal.

Muxes answer “which value proceeds?” They appear in ALU output selection, register input paths, operand forwarding, program-counter choice, and memory interfaces.

### Decoder: encoded choice becomes one-hot permission

A **decoder** maps an  $n$ -bit code to one of up to  $2^n$  output lines. A 2-to-4 decoder behaves like this when enabled:

| A1 | A0 | Y0 | Y1 | Y2 | Y3 |
|----|----|----|----|----|----|
| 0  | 0  | 1  | 0  | 0  | 0  |
| 0  | 1  | 0  | 1  | 0  | 0  |
| 1  | 0  | 0  | 0  | 1  | 0  |
| 1  | 1  | 0  | 0  | 0  | 1  |

The outputs are **one-hot**: exactly one is asserted. A register file can decode a destination number into one write-enable line. An instruction decoder can turn opcode fields into control conditions. Real decoders often also have an enable input.

## Encoder: asserted line becomes a code

An **encoder** performs the conceptual reverse:

one of several input lines → compact binary code

A plain encoder assumes only one input is asserted. If several can be active, the output is ambiguous. A **priority encoder** adds a rule—perhaps the highest-numbered asserted input wins—and often a valid output indicating that some request exists.

Keep the jobs distinct:

|         |                                            |
|---------|--------------------------------------------|
| mux     | many data values → one selected data value |
| decoder | compact code → one-hot control lines       |
| encoder | one-hot/request → compact code             |

## 7. Combinational versus sequential logic

A combinational circuit's settled output depends only on current inputs:

output = f(current inputs)

An adder, mux, decoder, and encoder are combinational. Remove or change the inputs, wait for propagation, and their outputs follow.

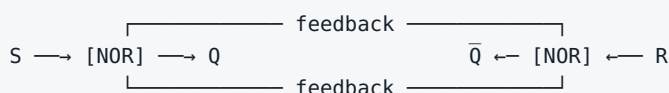
A sequential circuit's behavior also depends on stored state:

|            |                                    |
|------------|------------------------------------|
| next state | = f(current state, current inputs) |
| output     | = g(current state, current inputs) |

The word *sequential* does not merely mean “gates drawn in a sequence.” It means past activity can affect current behavior because some physical state persists.

## 8. Why feedback can create state

Feed a combinational output back into an input and the circuit can become self-reinforcing. Two cross-coupled NOR gates illustrate the idea:



In the idle condition  $S=0$ ,  $R=0$ , suppose  $Q=1$  and  $\bar{Q}=0$ . Each output helps force the other gate to preserve that arrangement. If Set is pulsed, the network moves to the set state; after Set returns inactive, feedback maintains it. The current input alone no longer determines the output. History matters.

This is an **SR latch**. For a NOR-based active-high form:

| S | R | next behavior                   |
|---|---|---------------------------------|
| 0 | 0 | hold previous state             |
| 1 | 0 | set $Q=1$                       |
| 0 | 1 | reset $Q=0$                     |
| 1 | 1 | forbidden/problematic condition |

The final row forces both outputs low, violating the normal complementary relationship. If S and R then return low nearly together, small physical delay differences can decide the resulting state; the circuit may also take time to resolve. Do not describe this row as a useful third stored value.

NAND-based SR latches commonly use active-low inputs and therefore have a different-looking table. Always state the circuit and signal polarity.

### Metastability is physical, not a third logic value

If storage timing requirements are violated, an internal node can temporarily hover between valid logical levels before resolving. Digital diagrams hide that analog behavior, but hardware cannot. Designers specify setup, hold, and recovery timing and use synchronizer structures when crossing asynchronous boundaries. No synchronous design can promise that arbitrary asynchronous input timing is physically harmless.

## 9. From SR to D storage

Most datapaths do not want callers to coordinate separate Set and Reset inputs. They want one data input:

```
store D when permitted; otherwise retain the old Q
```

### D latch: level-sensitive

A D latch uses gating so the forbidden SR request is not intentionally generated.

```
Enable = 1 → transparent: Q follows D after propagation delay  
Enable = 0 → closed: Q holds the last captured value
```

Calling the latch “transparent” matters. While Enable remains active, several changes at D may pass through to Q. It does not capture only at the instant Enable rises.

### D flip-flop: edge-triggered

An edge-triggered D flip-flop samples D around a specified clock transition, such as the rising edge. After its clock-to-Q delay, Q reflects the sampled value and remains stable until a later active edge.



```
before edge: D must satisfy setup time
at edge:      sampling event
after edge:   D must satisfy hold time
later:        Q changes after clock-to-Q delay
```

“At the edge” is a useful logical model, not a claim of zero-duration or perfect simultaneity. Real storage has an aperture defined by setup and hold requirements.

Conceptually:

```
D latch      level-sensitive while enabled
D flip-flop   edge-triggered around a clock transition
```

Some implementations construct an edge-triggered flip-flop from two opposite-phase latches; other circuit styles exist. The behavioral contract is what the surrounding datapath relies on.

## 10. Why a clock exists

Combinational signals do not settle simultaneously. A synchronous design creates a timing contract:

```
source register
  → clock-to-Q delay
  → combinational logic and routing delay
  → destination setup requirement
  → next active clock edge
```

The clock period must be long enough for the worst allowed path, plus timing margin and clock-distribution effects. Hold-time constraints impose a minimum path requirement after an edge.

The clock provides regular sampling opportunities and divides a large sequential computation into stages. It does not make propagation delay disappear, guarantee arbitrary asynchronous inputs are safe, or command all transistors to switch.

Clock networks themselves consume energy and need careful distribution. Real systems may gate clocks, use multiple clock domains, or include asynchronous blocks. “A CPU has a clock” is the beginning of timing analysis, not its end.

## 11. Registers: store a word, not just one bit

A **register** groups storage elements so several bits are captured under a shared control:

```
D3 D2 D1 D0
|  |  |  |
[FF][FF][FF][FF] ← shared clock
|  |  |  |
Q3 Q2 Q1 Q0
```

Four D flip-flops can hold a four-bit pattern. Practical registers may add reset, enable, scan, or other controls.

## Load enable is selection before storage

If a register should retain its value when `load=0` and capture `input` when `load=1`, place a mux before each D input:

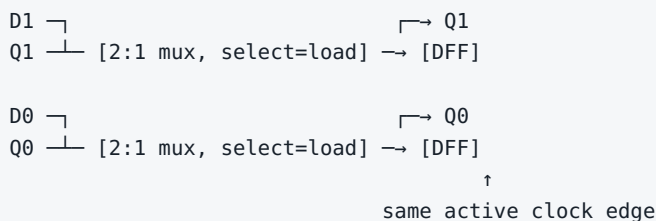
```
next = load ? input : current
```



Feedback supplies the hold value; the mux controls whether new or old data reaches D; the edge-triggered storage determines when Q may update.

## Design a two-bit register

Use two identical bit slices sharing `clock` and `load`:



Next-state equations:

```
Q1_next = load ? D1 : Q1
Q0_next = load ? D0 : Q0
```

State-transition table:

| current Q1Q0 | load | input D1D0 | next Q1Q0 at active edge |
|--------------|------|------------|--------------------------|
| 00           | 0    | 11         | 00                       |
| 00           | 1    | 10         | 10                       |
| 10           | 0    | 01         | 10                       |
| 10           | 1    | 11         | 11                       |
| 11           | 1    | 00         | 00                       |

Between active edges, changing D or load does not immediately change Q in this edge-triggered model. At an edge, both bits conceptually capture together, provided timing requirements are met.

Draw the complete state graph yourself. It has four nodes—`00`, `01`, `10`, `11`. With `load=0`, each node has a self-loop. With `load=1`, each node can transition to the input-selected destination.

## 12. Put the pieces around one addition

Now we can design a tiny register-transfer step:



The roles are sharply separated:

- registers preserve operand and result patterns;
- the adder continuously forms a combinational sum;
- a mux selects which candidate value reaches the destination D inputs;
- decoder/control logic supplies select and load signals;
- an active clock edge makes the chosen next state become stored state.

For  $3 + 5$ , operand registers expose `0011` and `0101`. Carries propagate until the adder settles at `1000`. Control selects the adder output and enables the result register. At the permitted edge—after timing constraints are satisfied—the result register captures `1000`.

This is already the shape of a CPU datapath. What is still missing is a control mechanism that interprets instruction bits, chooses registers and operations, and updates a program counter in the correct sequence. That is the next bridge.

## 13. Offline Digital Circuits Lab

Open:

[weekly/challenges/week-004-digital-circuits-lab.html](https://weekly/challenges/week-004-digital-circuits-lab.html)

Use it in this order:

1. **Half/full adder:** derive Sum and Carry before toggling inputs.
2. **Four-bit ripple adder:** enter two values, predict the result, then step across the visible carry chain.
3. **Mux/decoder:** predict the selected data and one-hot decoder line.
4. **D storage/register:** stage D and load values, then apply a clock edge. Observe that Q does not follow D between edges.
5. **Scored predictions:** answers remain hidden until each attempt.

The lab is a logical simulation. It does not model transistor analog behavior, metastability, or exact nanosecond timing. Its carry animation shows causal stage order, not a calibrated physical delay.

## 14. Evidence: Explain, Draw, Observe, Build

---

### Explain

Without notes:

1. Derive half-adder Sum and Carry from the four addition cases.
2. Explain why a full-adder needs `Cin` and why `Cout` is a majority function.
3. Explain what “carry propagation” means both logically and physically.
4. Distinguish mux, decoder, and encoder by the direction of information flow.
5. Explain why feedback lets past state affect present output.
6. Distinguish a level-sensitive D latch from an edge-triggered D flip-flop.
7. Explain what a clock coordinates without claiming that it switches every transistor.

### Draw

On one uninterrupted page, draw:

```
half-adder
  → full-adder
  → four-stage ripple adder
  → mux
  → two-bit enabled register
  → result path in a CPU-like datapath
```

Label `Cin`, `Cout`, `select`, `load`, `D`, `Q`, and the active clock edge.

### Observe

In the lab:

- find an addition whose carry crosses all four stages;
- record each `Cin`, Sum, and `Cout`;
- show that changing register D does not change Q before a clock edge;
- show one decoder input and its one-hot output;
- record one prediction that differed from observation and repair the mental model.

### Build

Complete both:

1. Build and document a four-bit ripple addition, including the carry chain.
2. Design the two-bit register above and write at least six state transitions, including hold, load, overwrite, and return-to-zero cases.

Passing the HTML score is useful evidence, but it cannot replace the drawing or verbal explanation.

---

## 15. Mastery checkpoint: 3 + 5, from physics to 8

---

Close everything and answer:

How are 3 and 5 transformed into electrical states and how does the machine produce and retain 8?

A complete trace must include:

1. unsigned encoding:  $3=0011$ ,  $5=0101$ ;
2. physical voltage/charge states implementing those logical bits;
3. one-bit full-adder behavior at all four positions;
4. carry sequence  $c1=1$ ,  $c2=1$ ,  $c3=1$ , final  $Cout=0$ ;
5. settled output  $1000$ ;
6. mux/control selection of the adder output;
7. capture by a destination register at an allowed edge;
8. interpretation of  $1000$  as unsigned 8;
9. the distinction between combinational propagation and clocked storage.

If you say “the clock makes the adder calculate,” repair the explanation. The input transition makes combinational circuitry respond. The clock coordinates when a storage element samples a result that must already satisfy timing requirements.

---

## Why? notebook

---

Write before looking back:

1. Why is a half-adder insufficient everywhere except a position with no incoming carry?
  2. Why can a ripple adder be logically correct but too slow for a target clock period?
  3. Why is a mux not merely an OR gate?
  4. Why does a decoder need a one-hot output, and what could use it?
  5. Why is a plain encoder ambiguous when several inputs are active?
  6. Why does feedback change a circuit from a current-input function into a stateful system?
  7. Why is the forbidden SR condition a physical timing concern, not just an inconvenient row in a table?
  8. Why can a D latch pass several changes while a D flip-flop captures at an edge?
  9. Why must data settle before an active edge and remain stable briefly after?
  10. Why does an enabled register contain both a selector and storage?
  11. Where does the value 8 “exist” physically, logically, and semantically?
  12. Which new machinery is still needed to turn this datapath into a CPU that follows instructions?
-

## Mental model at the end

physical voltage ranges

- logical operand bits
- combinational gates
- full-adder carry chain
- settled sum bits
- mux-selected next value
- edge-triggered register state
- later operations consume that state

The essential distinction is:

Combinational logic transforms present inputs after propagation delay. Sequential logic preserves state, so past events can influence the present. A synchronous datapath uses clocks to coordinate when storage samples combinational results; it does not make all transistors switch together.

## References used selectively

Local reading:

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 12–16, for binary addition, feedback, relays, selection, and the construction of larger digital systems. Local catalog path: `source-materials/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf`
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd edition, Chapter 3, especially the sections on combinational circuits, decoders, muxes, adders, storage elements, and finite-state behavior. Local catalog path: `source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

Historical and technical context:

- Claude E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits*, MIT Libraries: <https://dspace.mit.edu/handle/1721.1/11173>
- Computer History Museum, “The Transistor and Portable Electronics,” for the transition from relay/vacuum-tube constraints to solid-state switching: <https://www.computerhistory.org/siliconengine/invention-of-the-point-contact-transistor/>
- Nand2Tetris, sequential-logic and computer-construction materials: <https://www.nand2tetris.org/course>
- MIT OpenCourseWare, *Computation Structures*, digital logic and sequential circuit materials: <https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/>

These sources ground the circuit definitions and historical chain. The whiteboard derivation, 3 + 5 trace, state-transition exercise, and integrated datapath narrative are synthesized for this masterclass.